

# Multi-Insert in Logical Replication

## Batching the Apply Worker for Fun and Throughput

Andrei Lepikhov

pgEdge

09.05.2026

# Why This Matters

## 1. Async replication needs minimal lag.

Logical replication is asynchronous — every second of apply delay is a second of data you cannot read on the subscriber. Reducing per-tuple overhead shrinks that window.

## 2. Bulk influx is the typical pattern.

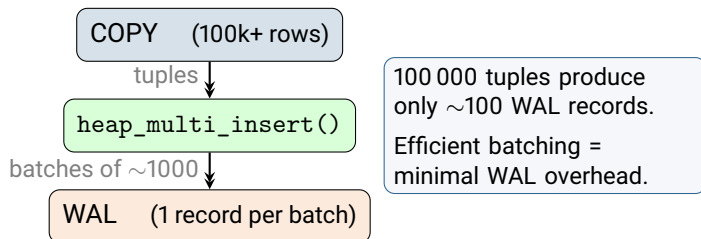
End-of-month batch loads, nightly ETL jobs — real workloads frequently arrive in bursts, not drips. The apply worker must handle millions of INSERTs at wire speed.

## 3. Multi-region means low-bandwidth channels.

Multi-cloud and cross-continent logical replication runs over constrained links. Every extra byte of WAL on the subscriber is wasted disk I/O *and* wasted recovery time when the link catches up.

# The Problem

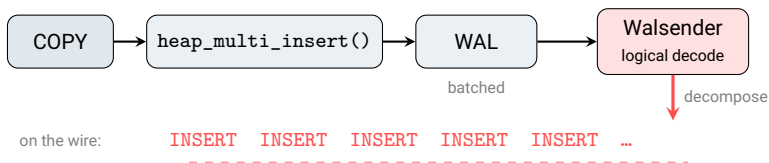
# How COPY Loads Data on the Publisher



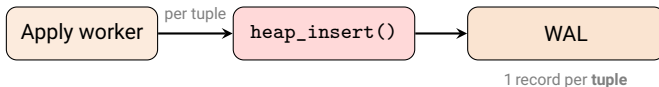
But what happens on the **subscriber**?

# How the Apply Worker Inserts (Before)

## Publisher



## Subscriber

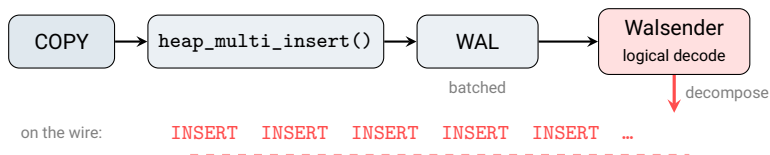


Publisher COPY writes 13 GB of WAL  $\Rightarrow$  subscriber generates **~18 GB**.  
WAL amplification on subscriber. Slow catch-up. Wasted I/O.

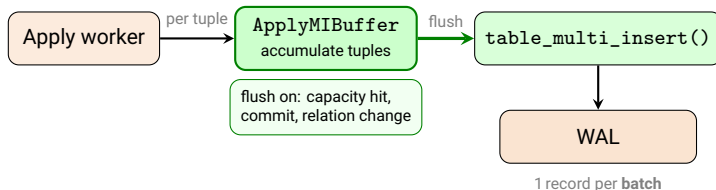
# The Idea

# Batch Inserts in the Apply Worker

## Publisher



## Subscriber





# Why No MULTI\_INSERT in the community Protocol?

- **MULTI\_INSERT is not a SQL operation.**

The logical replication protocol mirrors SQL-level changes: INSERT, UPDATE, DELETE. `heap_multi_insert()` is a storage-layer optimisation internal to COPY.

- **Protocol version negotiation.**

A new message type requires a version bump, publisher-subscriber negotiation, and fallback paths for older subscribers. High coordination cost.

- **Conflict resolution becomes harder.**

With per-row INSERT the subscriber can detect and resolve conflicts tuple by tuple. A MULTI\_INSERT batch is all-or-nothing.

- **Output plugin API is per-change.**

`LogicalDecodeChangeCB` receives one change. A batch API means a new callback type, new plugin API version, and backward compatibility across all output plugins.

# Enabling Multi-Insert

-- Automatic slot creation:

```
CREATE SUBSCRIPTION mysub  
  CONNECTION 'host=publisher port=5432 dbname=mydb'  
  PUBLICATION mypub  
  WITH (multi_insert = on, streaming = off);
```

-- With a manually created slot:

-- 1. On publisher (replication connection):

```
CREATE_REPLICATION_SLOT mysub_slot LOGICAL pgoutput;
```

-- 2. On subscriber:

```
CREATE SUBSCRIPTION mysub  
  CONNECTION 'host=publisher port=5432 dbname=mydb'  
  PUBLICATION mypub  
  WITH (  
    create_slot = false,  
    slot_name = 'mysub_slot',  
    multi_insert = on,  
    streaming = off  
  );
```

multi\_insert is always a subscription option – no protocol changes.



# Implementation

# ApplyMIBuffer – The Batching Engine

## Lifecycle:

- 1 Single static buffer, initialized on first INSERT for an eligible relation
- 2 Accumulate tuples in slot array
- 3 **Intermediate flush:** capacity hit  $\rightarrow$  call `table_multi_insert()`, clear slots, reuse buffer
- 4 **Final flush:** commit, relation change, or non-INSERT event  $\rightarrow$  flush and reinitialize for the next relation

## Slot management:

- Fixed array of 10 000 `TupleTableSlot*`, lazily allocated one per tuple
- Slots reused across flushes (only `nslots` resets to 0)
- Flush when `nslots`  $\geq$  10 000 or `cum_bytes`  $\geq$  8 MB

# Table Eligibility — Who Gets Batched?

A table is eligible for multi-insert if it has **none** of:

- BEFORE/INSTEAD OF triggers
- Row-level security
- CHECK constraints
- Stored generated columns
- Exclusion constraints
- Deferrable UNIQUE/PK
- Invalid or unready indexes
- Non-plain table (partitioned, foreign)

Ineligible tables **silently fall back** to per-tuple insert.

Stricter than COPY: LR bypasses the executor, so constraints that COPY validates at runtime must be rejected statically.

# Benchmarks

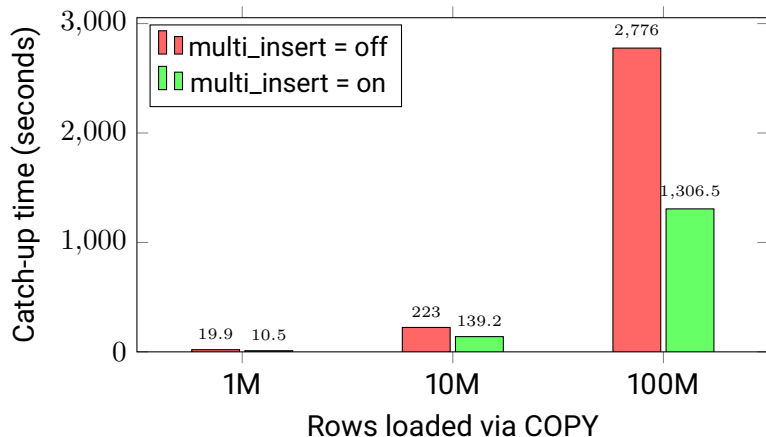
# Benchmark Setup



```
bench_copy (  
  id    bigint PRIMARY KEY,  
  val   double precision,  
  payload text,  
  ts    timestampz  
)
```

- Three dataset sizes: 1M, 10M, 100M rows
- `multi_insert = true` VS `false`, `streaming = off`

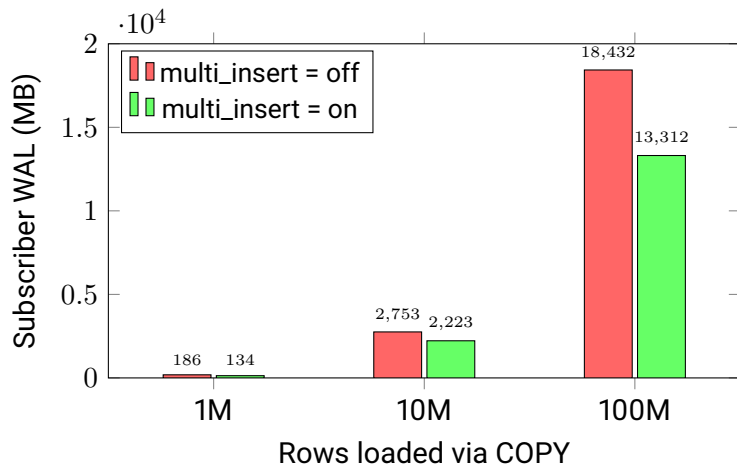
# Replication Catch-Up Time



**1.6× – 2.1× faster** replication catch-up across all sizes.



# Subscriber WAL Generated



**19% – 28% less WAL** on the subscriber. At 100M rows: 18 GB → 13 GB.

# Challenges

# Challenges & Limitations

- **Conflicts and errors cause fallback to single-insert.**

A unique violation or other error during batch flush aborts the remote transaction. Standard per-row conflict resolution (`disable_on_error`, `SKIP`) is bypassed.

- **Streaming must incorporate batching.**

Currently `multi_insert` requires `streaming = off`. Combining both is needed for a production-ready version.

- **Parallel apply workers excluded.**

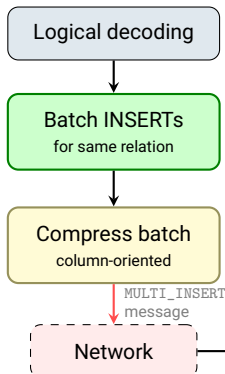
Multi-insert is disabled inside parallel apply workers.

- **Per-transaction disable on first ineligible relation.**

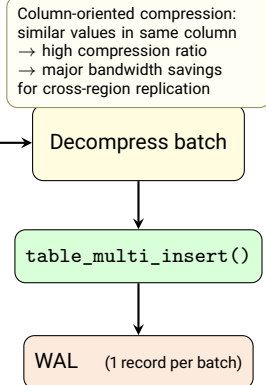
If any relation fails the eligibility check, batching is disabled for the *entire* remaining transaction, even for other eligible tables.

# Future: MULTI\_INSERT Protocol Message

## Publisher (walsender)



## Subscriber (apply worker)



# Summary

- Logical replication apply worker now supports batched inserts
- **2× faster** replication catch-up on bulk loads
- **28% less WAL** generated on the subscriber
- Subscriber-side only — zero publisher changes
- Opt-in via `multi_insert = on`, graceful fallback



# Questions?